```
  def chatty_string = "I'm not done"
end

class DefinedStatus < Status
  def to_s = "defined"

  def chatty_string = "I'm not even started"
end
```

Now, if we want to get at that particular chatty string above, rather than having to do the case expression explicitly, we can write something like this:

```
Status.for(task.status).chatty_string
```

The case logic is now behind the scenes, in our Status.for method. Once we call it, we know what kind of status we have, and each kind of status knows its own behavior, so we can now call chatty_string directly on the status. More to the point, once we call Status.for, we don't need to have that case logic again; we've removed the potential duplication.

The done? method is defined in the parent class as being false, which is fine for the StartedStatus and DefinedStatus classes, but incorrect for the DoneStatus class, which therefore overrides done? to the correct value—true—for that class. There is no default for the chatty_string method though, so the parent class throws an exception if it is somehow called. This is a signal that all the subclasses must define this method.

The is a common idiom when using subclassing. A parent class assumes that it will be subclassed and calls a method that it expects its children to implement. This allows the parent to take on the brunt of the processing but to invoke what are effectively hook methods in subclasses to add application-level functionality. As we'll see at the end of this chapter, just because this idiom is common doesn't make it good design.

So, instead, let's look at *mixins*, a different way of sharing functionality in Ruby code. But, before we look at mixins, we'll need to get familiar with Ruby *modules*.

In Ruby, a module can do everything that a class can do, except create instances. It turns out, that even without creating instances, it still can be useful to group related methods and data together. Let's explore how.

Modules

Modules are a way of grouping together methods, classes, and constants. Modules give you two major benefits:

- Modules provide a namespace and prevent name clashes.
- Modules can be included in other classes, a facility known as a *mixin*.

Namespaces

As you start to write bigger Ruby programs, you'll find yourself producing chunks of reusable code—libraries of related routines that are generally applicable. You'll want to break this code into separate files so the contents can be shared among different Ruby programs.

Often this code will be organized into classes, so you'll probably stick a class into each file. However, there are times when you want to group things together that don't naturally form